Số học cơ bản - Phần 2

Lũy thừa nhị phân, Modulo & Các thuật toán Sàng

Chuyên Đề Thuật Toán

Mục lục

1. Lũy thừa nhị phân (Binary Exponentiation)
2. Modulo - Các phép toán đồng dư & Nghịch đảo
3. Thuật toán Sàng nguyên tố (Sieve of Eratosthenes)
4. Phát triển Sàng: Số lượng và Tổng ước
5. Sàng SPF & Phân tích thừa số nguyên tố
6. Số chính phương & Cấu trúc thừa số

Lũy thừa nhị phân - Bài toán

Đặt vấn đề

Tính $a^b \pmod M$ với $a, b \leq 10^{18}$ và $M \leq 10^9 + 7$.
Nếu dùng vòng lặp 'for' thông thường, độ phức tạp là $O(b)$, quá chậm và sẽ gây ra **Time Limit Exceeded (TLE)**.

Ý tưởng thuật toán

Thay vì nhân a với chính nó b lần, ta biểu diễn b dưới dạng nhị phân.
Ví dụ: $b = 13 = 1101_2 = 2^3 + 2^2 + 2^0 = 8 + 4 + 1$.
Suy ra: $a^{13} = a^8 \cdot a^4 \cdot a^1$.

Phân tích trên tập bit

Duyệt qua các bit của b

- Tại mỗi bước, ta duy trì biến a tự bình phương lên ($a \to a^2 \to a^4 \to a^8 \dots$).
- Kiểm tra bit cuối cùng của b: Nếu là 1, nhân a hiện tại vào kết quả *res*.
- Dịch bit b sang phải ($b \gg 1$) để xét bit tiếp theo.

Mẹo (Tips)

Kiểm tra bit cuối của b bằng thao tác bitwise: `b & 1`.
Chia b cho 2 bằng thao tác: `b >>= 1`.
Điều này nhanh hơn các phép toán số học `%` và `/`.

Mô phỏng Lũy thừa nhị phân (Tính 3^{13})

Bước	b **(nhị phân)**	b **(& 1)**	$a \leftarrow a^2$	**Kết quả (*res*)**
0 (Init)	1101	-	3^1	1
1	1101	1	3^2	$1 \cdot 3^1$
2	110	0	3^4	Bỏ qua, vẫn là 3^1
3	11	1	3^8	$3^1 \cdot 3^4 = 3^5$
4	1	1	3^{16}	$3^5 \cdot 3^8 = 3^{13}$

Kết quả cuối cùng: $res = 3^{13}$

Cài đặt Lũy thừa nhị phân

Listing 1: Binary Exponentiation

```
long long binpow(long long a, long long b, long long M) {
    long long res = 1;
    a %= M;
    while (b > 0) {
        if (b & 1) { // Neu bit cuoi la 1
            res = (res * a) % M;
        }
        a = (a * a) % M; // Binh phuong a o moi buoc
        b >>= 1;         // Dich bit sang phai (b /= 2)
    }
    return res;
}
```

Độ phức tạp

Thời gian: $O(\log b)$ do số lượng bit của b là $\approx \log_2(b)$.
Không gian: $O(1)$.

Các phép đồng dư cơ bản

Công thức cộng, trừ, nhân

Với mọi a, b và module M:

- $(a + b) \pmod M = ((a \pmod M) + (b \pmod M)) \pmod M$
- $(a - b) \pmod M = ((a \pmod M) - (b \pmod M) + M) \pmod M$
- $(a \times b) \pmod M = ((a \pmod M) \times (b \pmod M)) \pmod M$

Sai lầm thường gặp ở phép trừ

Trong C++, phép chia lấy dư của số âm có thể trả về số âm (VD: `-5 % 3 = -2`). Để luôn lấy modulo dương chuẩn toán học, luôn phải **cộng thêm** M trước khi `% M` như công thức trên.

Phép chia và Nghịch đảo Modulo

Ngộ nhận toán học

$\left(\frac{A}{B}\right) \pmod M \neq \frac{A \pmod M}{B \pmod M}$

Ý tưởng thuật toán

Phép chia $\frac{A}{B} \pmod M$ tương đương với nhân A với **nghịch đảo modulo** của B.
Tức là: $\frac{A}{B} \equiv A \times B^{-1} \pmod M$.
Vậy B^{-1} là số x thỏa mãn: $B \times x \equiv 1 \pmod M$.

Chứng minh Định lý Fermat nhỏ

Nếu M là một số nguyên tố và B không chia hết cho M, ta có:
$B^{M-1} \equiv 1 \pmod M$
Nhân 2 vế với B^{-1}, ta được: $B^{-1} \equiv B^{M-2} \pmod M$.

Minh họa trực tiếp: Tại sao cần Nghịch đảo Modulo?

Bài toán: Tính $\left(\frac{33}{3}\right) \pmod 7$

1. Kết quả thực tế (Tính nhẩm):
Ta có $\frac{33}{3} = 11 \Rightarrow 11 \pmod 7 = \mathbf{4}$.

Áp dụng Fermat nhỏ cho nghịch đảo Modulo

Nếu lấy: $\frac{33 \pmod 7}{3 \pmod 7} = \frac{5}{3} \approx 1.66$ $\Rightarrow$ **Sai hoàn toàn!**
(Trong số học modulo không có khái niệm số thập phân).
Thay vì chia, ta nhân A với nghịch đảo modulo của B:

- Lấy dư A: $33 \equiv 5 \pmod 7$.
- Lấy dư B: $3 \equiv 3 \pmod 7$.
- Tìm nghịch đảo của B: $3^{-1} \equiv 3^{7-2} \equiv 3^5 \equiv 243 \equiv \mathbf{5} \pmod 7$.
- Phép tính trở thành: $(A \times B^{-1}) \pmod 7 = (5 \times \mathbf{5}) \pmod 7 = 25 \pmod 7 = \mathbf{4}$.

Cài đặt Nghịch đảo Modulo

Phương pháp tính

Để tính $(A/B) \pmod M$ với M là số nguyên tố, ta chỉ cần gọi hàm `binpow(B, M-2, M)` để lấy nghịch đảo, sau đó nhân với A.

Listing 2: Modular Inverse & Division

```
// Luu y: M phai la so nguyen to
long long modInverse(long long b, long long M) {
    return binpow(b, M - 2, M);
}

long long modDivide(long long a, long long b, long long M) {
    a = a % M;
    long long inv = modInverse(b, M);
    return (a * inv) % M;
}
```

Ý tưởng Sàng nguyên tố

Nguyên lý loại trừ

Để tìm tất cả các số nguyên tố từ 1 đến N:

1. Bắt đầu, giả sử tất cả các số từ 2 đến N đều là số nguyên tố.
2. Duyệt i từ 2, nếu i là số nguyên tố, ta **loại bỏ tất cả các bội số** của i $(2i, 3i, 4i \dots)$ vì chúng chắc chắn là hợp số.
3. Tiếp tục với số nguyên tố tiếp theo chưa bị loại bỏ.

Tối ưu

Vòng lặp ngoài chỉ cần chạy đến $\sqrt{N}$.
Vòng lặp trong (loại bội số) có thể bắt đầu từ $i \times i$ (thay vì $2 \times i$), vì các bội nhỏ hơn i đã bị sàng bởi các số nguyên tố nhỏ hơn.

Mô phỏng Sàng nguyên tố ($N = 20$)

Bảng trạng thái từ 2 đến 20:

	2 (NT)	**3 (NT)**	4 (Bội của 2)	**5 (NT)**
6 (Bội 2,3)	**7 (NT)**	8 (Bội của 2)	9 (Bội của 3)	10 (Bội 2,5)
11 (NT)	12 (Bội 2,3)	**13 (NT)**	14 (Bội 2,7)	15 (Bội 3,5)
16 (Bội của 2)	**17 (NT)**	18 (Bội 2,3)	**19 (NT)**	20 (Bội 2,5)

- **Bước 1 ($i = 2$):** Đánh dấu 2 là nguyên tố, gạch bỏ 4, 6, 8, 10, 12, 14, 16, 18, 20.
- **Bước 2 ($i = 3$):** Đánh dấu 3 là nguyên tố, gạch bỏ 9, 15 (6, 12, 18 đã bị gạch).
- **Bước 3 ($i = 4$):** Đã bị gạch, bỏ qua.
- **Bước 4 ($i = 5$):** $5 > \sqrt{20}$ (Dừng thuật toán).
- **Các số màu xanh còn lại chính là số nguyên tố!**

Cài đặt Sàng nguyên tố

Listing 3: Sieve of Eratosthenes

```
const int MAX = 1e6 + 5;
bool is_prime[MAX];

void sieve() {
    // Khoi tao mang toan true
    for (int i = 0; i < MAX; i++) is_prime[i] = true;
    is_prime[0] = is_prime[1] = false;

    for (int i = 2; i * i < MAX; i++) {
        if (is_prime[i]) {
            // Loai bo cac boi so, bat dau tu i * i
            for (int j = i * i; j < MAX; j += i) {
                is_prime[j] = false;
            }
        }
    }
}
```

Độ phức tạp thời gian: $O(N \log \log N)$ (Gần như tuyến tính).

Sàng Số lượng và Tổng ước

Ý tưởng thuật toán Sàng trên mọi số

Tương tự nguyên lý sàng nguyên tố, thay vì chỉ đánh dấu hợp số, ta dùng ý tưởng duyệt bội số để đếm số lượng hoặc tính tổng các ước số cho **mọi số** từ 1 đến N.

Thuật toán

- Duyệt i từ 1 đến N.
- i chắc chắn là ước của các số: $i, 2i, 3i, 4i \ldots$
- Với mỗi bội số $j = k \cdot i$, ta cập nhật mảng kết quả: `count[j] += 1` hoặc `sum[j] += i`.

Ghi chú

Khác với sàng nguyên tố, ta duyệt i bắt đầu từ 1 và vòng lặp trong bắt đầu từ chính i (chứ không phải i^2). Không có lệnh điều kiện `if (is_prime[i])`.

Cài đặt Sàng Số lượng và Tổng ước

Listing 4: Divisor Sieve

```
const int MAX = 1e6 + 5;
int num_divs[MAX];  // So luong uoc
long long sum_divs[MAX];  // Tong cac uoc

void divisor_sieve() {
    for (int i = 1; i < MAX; i++) {
        for (int j = i; j < MAX; j += i) {
            num_divs[j]++;       // j co mot uoc la i
            sum_divs[j] += i;    // Cong i vao tong uoc cua j
        }
    }
}
```

Độ phức tạp

Tổng số lần lặp: $\sum_{i=1}^{N} \frac{N}{i} = N\left(1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{N}\right)$.
Theo chuỗi Harmonic, tổng này xấp xỉ $\ln N$.
Độ phức tạp thời gian: $O(N \log N)$.

Sàng SPF (Smallest Prime Factor)

Định nghĩa

SPF (Smallest Prime Factor) của số x là ước nguyên tố nhỏ nhất của x.
Ví dụ: SPF(15) = 3, SPF(20) = 2, SPF(17) = 17 (do 17 là số nguyên tố).

Tại sao cần SPF?

Thuật toán phân tích thừa số nguyên tố thông thường duyệt tới $\sqrt{N}$, mất $O(\sqrt{N})$ cho MỘT truy vấn.
Nếu có Q truy vấn ($Q \leq 10^5$), tổng thời gian $O(Q\sqrt{N})$ sẽ bị TLE.
Sử dụng mảng SPF được chuẩn bị trước bằng thuật toán Sàng, ta có thể phân tích thừa số nguyên tố trong thời gian $O(\log N)$ mỗi truy vấn!

Cài đặt Sàng SPF

```
1  int spf[MAX];
2
3  void sieve_spf() {
4      for (int i = 1; i < MAX; i++)
5          spf[i] = i; // B/dieu, uoc nho nhat la
               chinh no
6
7      for (int i = 2; i * i < MAX; i++) {
8          if (spf[i] == i) { // Neu i la so NT
9              for (int j = i * i; j < MAX; j += i)
                   {
10                 // Chi cap nhat neu j chua duoc
                       gan
11                 if (spf[j] == j)
12                     spf[j] = i;
13             }
14         }
15     }
16 }
```

Mẹo (Tips)

Dòng `if (spf[j] == j)` cực kỳ quan trọng. Nó đảm bảo ta ghi nhận ước nguyên tố NHỎ NHẤT đầu tiên chạm tới j.
VD: $j = 12$ sẽ được cập nhật $spf[12] = 2$ trước. Khi $i = 3$ chạy tới, nó không ghi đè vì $spf[12] \neq 12$.

Thuật toán phân tích thừa số với SPF

Quá trình phân tích x

- Liên tục lấy x chia cho $spf[x]$ cho đến khi $x = 1$.
- Tập các số $spf[x]$ lấy ra chính là cấu trúc thừa số nguyên tố của x.

Listing 5: O(log N) Factorization

```
vector<int> factorize(int x) {
    vector<int> res;
    while (x != 1) {
        res.push_back(spf[x]);
        x = x / spf[x];
    }
    return res;
}
```

Mô phỏng phân tích SPF của số 60

Phân tích số 60 liên tục bằng bảng mảng SPF

Giá trị hiện tại (x)	SPF(x)	Thao tác tiếp theo	Thừa số sinh ra
60	2	$x \leftarrow 60/2 = 30$	**2**
30	2	$x \leftarrow 30/2 = 15$	**2**
15	3	$x \leftarrow 15/3 = 5$	**3**
5	5	$x \leftarrow 5/5 = 1$	**5**
1	-	**Dừng thuật toán**	-

Tổng kết

Ta thu được các thừa số sinh ra: $60 = 2 \times 2 \times 3 \times 5 = \mathbf{2^2 \cdot 3^1 \cdot 5^1}$.
Chỉ mất đúng 4 bước vòng lặp (`while`) thay vì duyệt bằng vòng `for`.

Định lý cơ bản về số chính phương

Một số nguyên dương N là số chính phương khi và chỉ khi trong phân tích thừa số nguyên tố của N:

$$N = p_1^{e_1} \cdot p_2^{e_2} \cdots p_k^{e_k}$$

thì **mọi số mũ e_i đều là số chẵn**.

Chứng minh

Nếu $N = A^2$, gọi phân tích của A là $p_1^{x_1} \cdots p_k^{x_k}$.
Khi bình phương lên, $N = A^2 = (p_1^{x_1} \cdots p_k^{x_k})^2 = p_1^{2x_1} \cdot p_2^{2x_2} \cdots p_k^{2x_k}$.
Rõ ràng mọi số mũ $2x_i$ luôn chia hết cho 2 (là số chẵn).

Minh họa cấu trúc Số chính phương

$$144 = 12^2$$

- **Bước 1: Phân tích 144 ra thừa số nguyên tố**

$$144 = 2 \times 2 \times 2 \times 2 \times 3 \times 3$$

- **Bước 2: Nhóm các thừa số giống nhau thành cấu trúc mũ**

$$144 = (2 \times 2 \times 2 \times 2) \times (3 \times 3) = \mathbf{2^4} \times \mathbf{3^2}$$

- **Bước 3: Nhận xét kết quả**
  Các số mũ của 2 và 3 lần lượt là **4** và **2**. Chúng đều là số chẵn.
  $\Rightarrow$ Mọi thừa số đều có thể ghép thành **từng cặp hoàn hảo**.

Ứng dụng trong thi đấu

Bài toán điển hình

Cho số nguyên dương X. Tìm số nguyên dương Y nhỏ nhất sao cho $X \times Y$ là một số chính phương.

Thuật toán

1. Phân tích X ra thừa số nguyên tố.
2. Kiểm tra số mũ của từng nguyên tố p_i.
3. Nếu p_i có số mũ **lẻ**, ta cần nhân thêm một p_i vào Y để số mũ trở thành chẵn.
4. Kết quả Y chính là tích của tất cả các nguyên tố có số mũ lẻ trong X.

Tổng kết

Sử dụng kết hợp **Sàng SPF** và **Lý thuyết số mũ chẵn** để giải quyết bài toán này trong thời gian cực nhanh với nhiều truy vấn!

Cảm ơn các bạn đã lắng nghe!